

Ingeniería de soluciones con AI

Semestre 2026-2

Docente:
Francisco Macaya

Noticias: Agentic Architecture as small, honest programs.

Medium

Search

Get app

Write

Sign up

Sign in



Beyond the workflow: agentic Architecture as small, honest programs

How we rebuilt our AI workflow engine around agents and tools that own their behavior — and why “who answers the user” became a first-class design question.



Darío Andrés Muñoz Prudent

Follow

7 min read · Apr 1, 2026



6

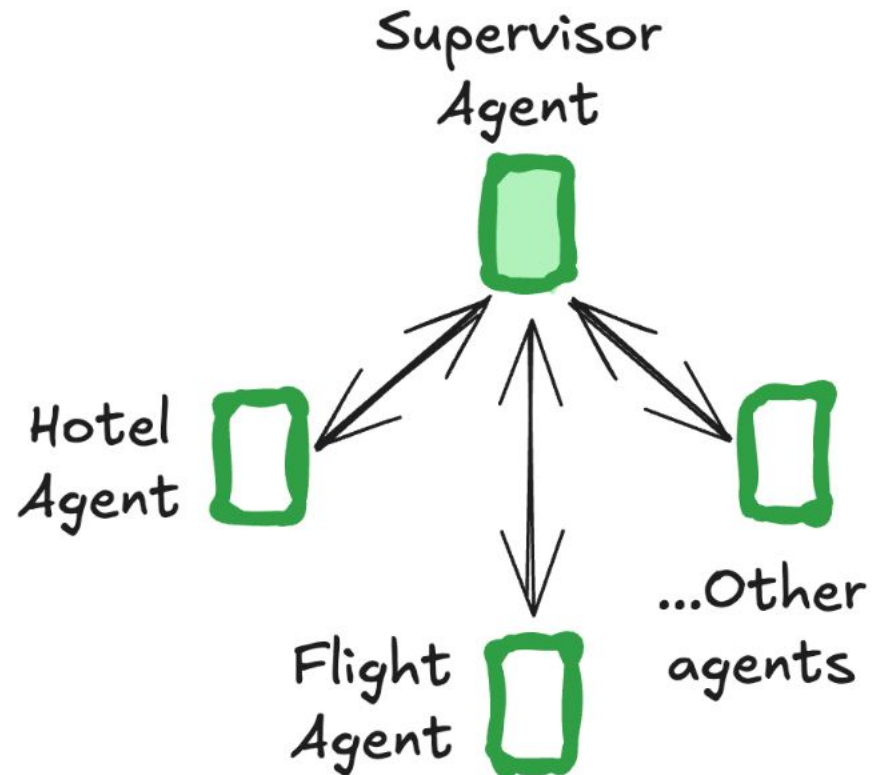


Nota: <https://ai.plainenglish.io/beyond-the-workflow-agentic-architecture-as-small-honest-programs-24e6b2facd3a>



*¿Quién es el?
Jawed Karim, “Me at the zoo”.*

Multi – Agents system: Cuando lo simple no es suficiente.



“In 2025, the models out there are extremely intelligent. But even the smartest human won’t be able to do their job effectively without the context of what they’re being asked to do. “Prompt engineering” was coined as a term for the effort needing to write your task in the ideal format for a LLM chatbot. “Context engineering” is the next level of this. It is about doing this automatically in a dynamic system. It takes more nuance and is effectively the #1 job of engineers building AI agents.”

Multi – Agents system: 4 patrones para mirar.

Pattern	How it works
<u>Subagents</u>	A main agent coordinates subagents as tools. All routing passes through the main agent, which decides when and how to invoke each subagent.
<u>Handoffs</u>	Behavior changes dynamically based on state. Tool calls update a state variable that triggers routing or configuration changes, switching agents or adjusting the current agent's tools and prompt.
<u>Skills</u>	Specialized prompts and knowledge loaded on-demand. A single agent stays in control while loading context from skills as needed.
<u>Router</u>	A routing step classifies input and directs it to one or more specialized agents. Results are synthesized into a combined response.
<u>Custom workflow</u>	Build bespoke execution flows with <u>LangGraph</u> , mixing deterministic logic and agentic behavior. Embed other patterns as nodes in your workflow.

SubAgents: Orquestación centralizada.

¿Cómo funcionan?:

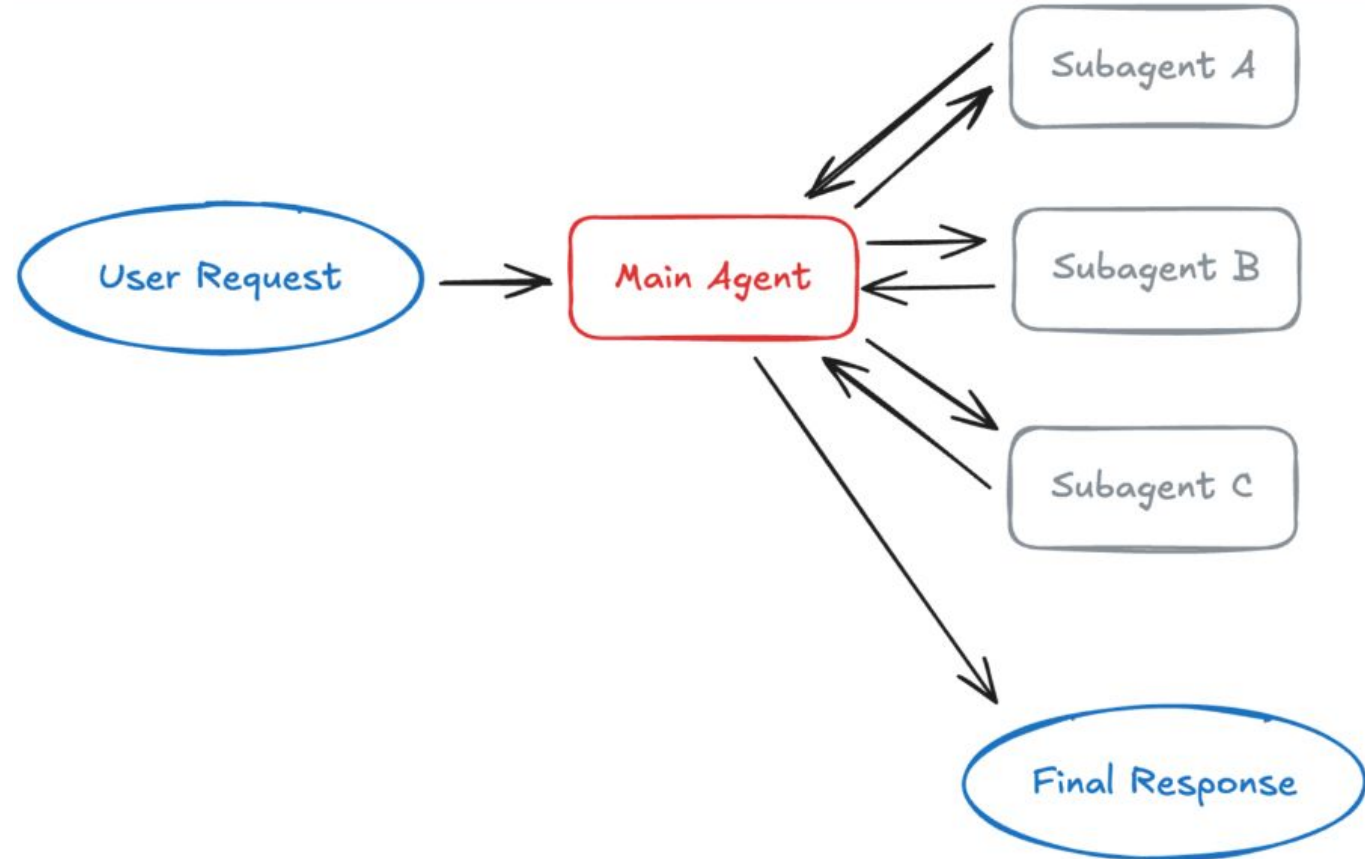
El agente principal decide que subagente invocar, que información proporcionar y como combinar los resultados. El agente principal puede invocar a múltiples agentes en paralelo.

Ideal:

Aplicaciones con múltiples dominios distintos donde se necesita un control centralizado del flujo de trabajo y los subagentes no necesitan conversar directamente con los usuarios.

¿Trade-off?:

Añade una llamada al modelo adicional por interacción, ya que los resultados deben volver a pasar por el agente principal.



Skills: Divulgación progresiva.

¿Cómo funcionan?:

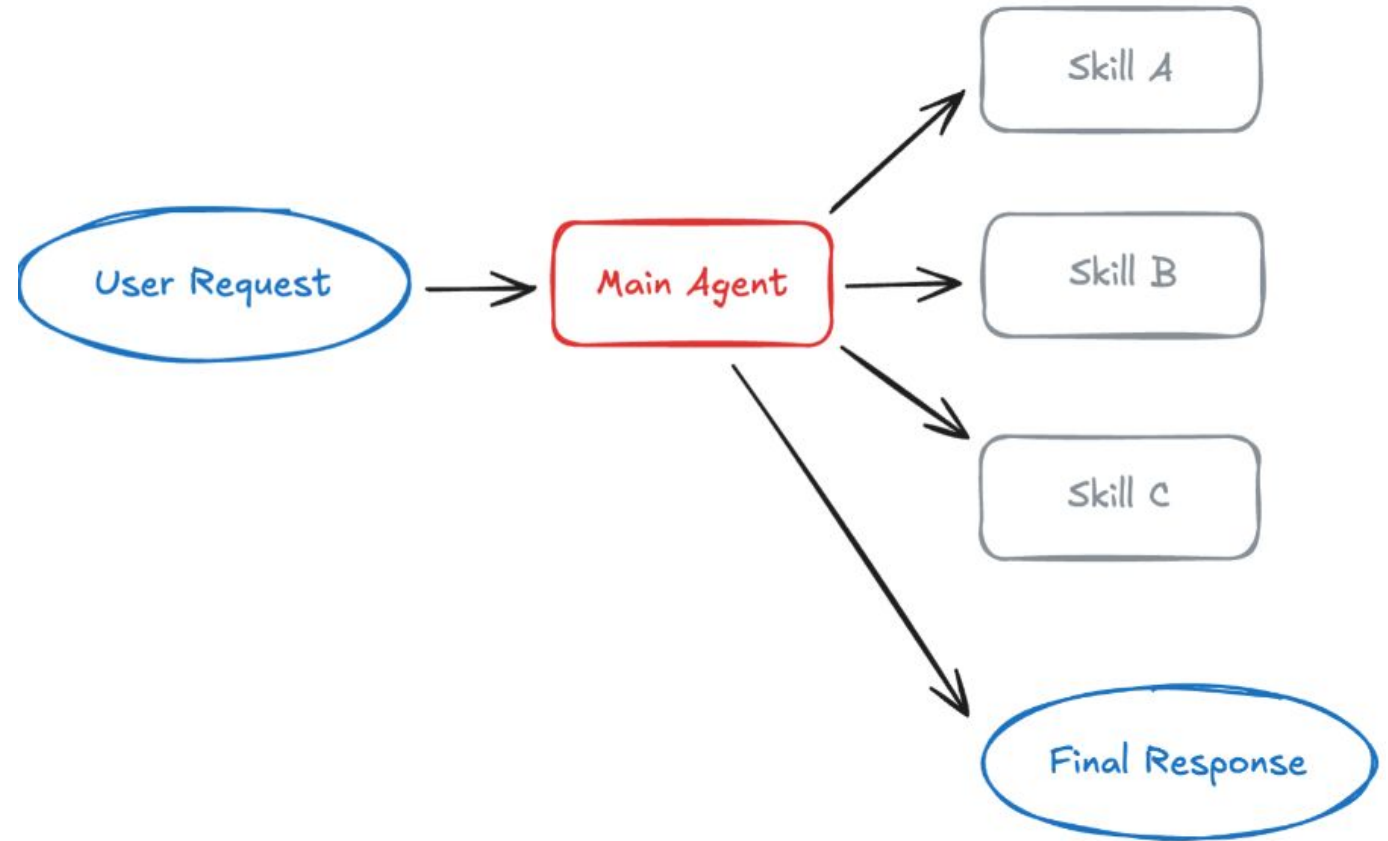
Las habilidades son especializaciones basadas principalmente en prompts, empaquetadas como directorios que contienen instrucciones, scripts y recursos. Al iniciarse, el agente solo conoce los nombres y descripciones de las habilidades. Cuando una habilidad se vuelve relevante, el agente carga su contexto completo.

Ideal:

Agentes únicos con muchas especializaciones posibles, situaciones donde no se necesita imponer restricciones entre capacidades, o equipos distribuidos donde distintos equipos mantienen diferentes habilidades.

¿Trade-off?:

El contexto se acumula en el historial de la conversación a medida que se cargan las habilidades, lo que puede generar una acumulación excesiva de tokens en llamadas posteriores.



Handoffs: Transiciones impulsadas por el estado.

¿Cómo funcionan?:

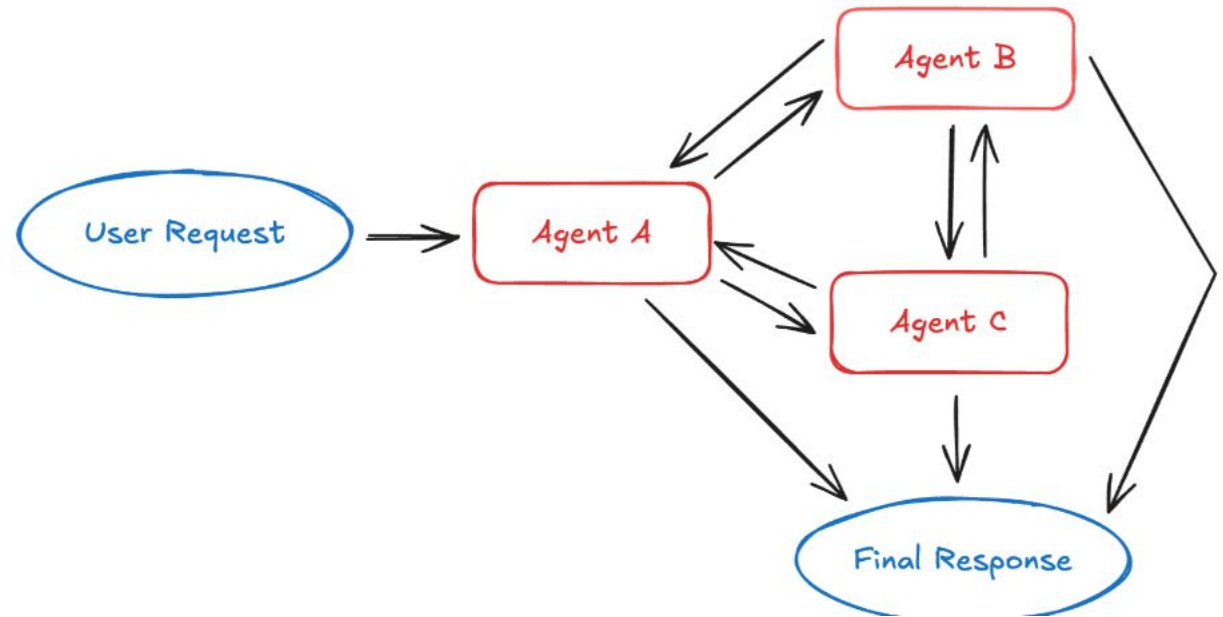
Cuando un agente invoca una herramienta de transferencia, actualiza el estado que determina cuál será el próximo agente en activarse. Esto puede implicar cambiar a un agente diferente o modificar el prompt del sistema y las herramientas disponibles del agente actual.

Ideal:

Flujos de atención al cliente que recopilan información por etapas, experiencias conversacionales en múltiples fases, o cualquier escenario que requiera restricciones secuenciales donde las capacidades se desbloquean solo después de cumplir ciertas condiciones previas.

¿Trade-off?:

Es más dependiente del estado que otros patrones, lo que requiere una gestión cuidadosa del mismo.



Handoffs: Asistente de un banco.

Internamente, el agente interactúa y cambia a otros agentes sin que el usuario expresamente haya guiado a esos nodos.

Situación inicial: El usuario escribe "Quiero hacer una transferencia"

El agente activo es el **Agente de Bienvenida**. Su único trabajo es identificar qué quiere el usuario. Cuando detecta que quiere hacer una transferencia, llama a una herramienta llamada `transferir_a_verificacion()`.

¿Qué pasa internamente cuando se llama esa herramienta?

Se actualiza una variable de estado:

```
estado_actual = "verificacion_identidad"
```

Eso dispara un cambio: ahora el sistema activa el **Agente de Verificación**, que tiene un prompt diferente, y solo tiene acceso a herramientas como `verificar_rut()` o `verificar_clave()`. No puede hacer transferencias todavía.

El usuario pasa la verificación. El agente llama a `transferir_a_operaciones()` y el estado cambia:

```
estado_actual = "operaciones_bancarias"
```

Ahora se activa el **Agente de Operaciones**, que sí tiene acceso a `ejecutar_transferencia()`. Esta herramienta no estaba disponible antes porque no se habían cumplido las condiciones previas.

Router: Dirige y ejecuta en paralelo.

¿Cómo funcionan?:

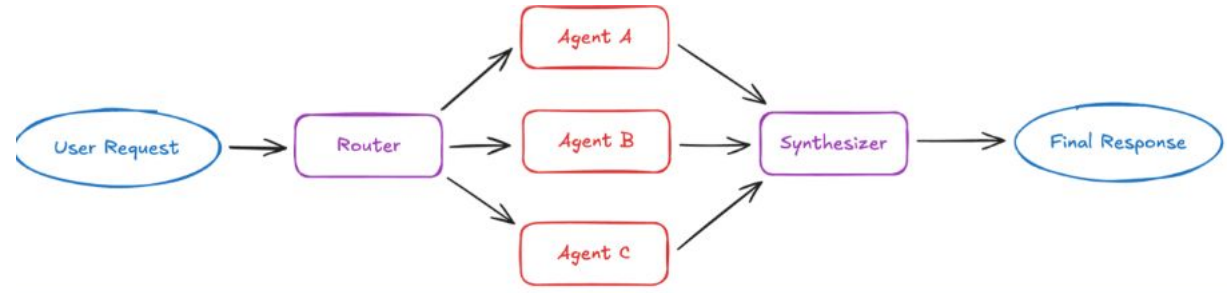
El enrutador descompone la consulta, invoca cero o más agentes especializados en paralelo y sintetiza los resultados en una respuesta coherente.

Ideal:

Aplicaciones con verticales distintas (dominios de conocimiento separados), escenarios que requieren consultas a múltiples fuentes en paralelo, o situaciones donde se necesita sintetizar resultados de múltiples agentes.

¿Trade-off?:

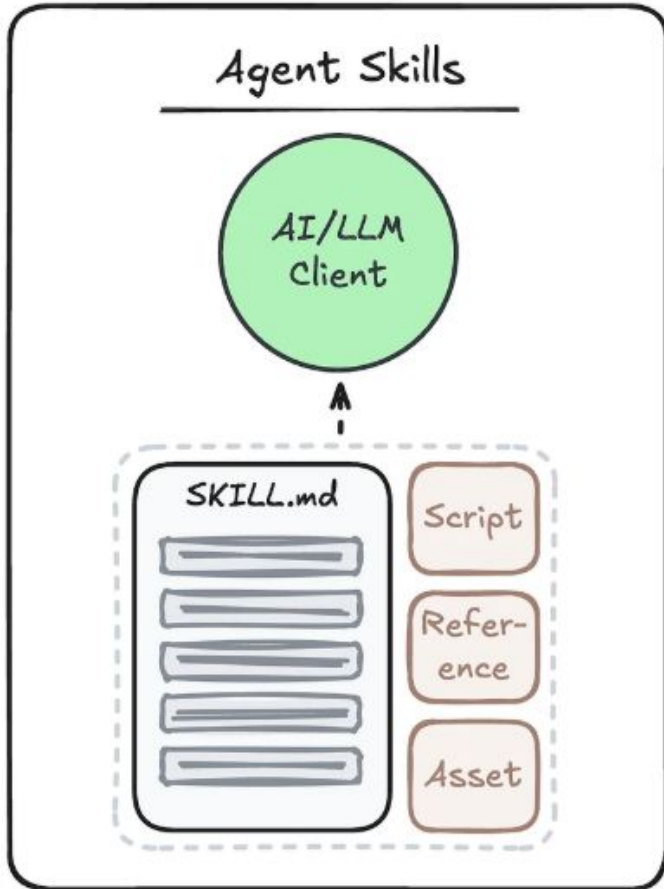
El diseño sin estado garantiza un rendimiento consistente por solicitud, pero genera una sobrecarga repetida de enrutamiento si se necesita historial de conversación.



¿Cuál elegir?: Depende.

YOUR REQUIREMENTS	PATTERN
Multiple distinct domains (calendar, email, CRM), need parallel execution	Subagents
Single agent with many possible specializations, lightweight composition	Skills
Sequential workflow with state transitions, agent converses with user throughout	Handoffs
Distinct verticals, query multiple sources in parallel and synthesize results	Router

Skills: Paquetes de conocimientos y habilidades.



Las **skills** (habilidades) agrupan conocimientos especializados —como flujos de trabajo, mejores prácticas, **scripts**, documentos de referencia y plantillas— en directorios reutilizables. El agente recibe un resumen del contenido al iniciarse, y solo descubre y lee los archivos incluidos cuando resulta pertinente.

Level	What loads	When
1. Metadata	<code>name</code> and <code>description</code> from <code>SKILL.md</code> <code>frontmatter</code>	Agent startup, for every configured skill
2. Instructions	Full <code>SKILL.md</code> body	When the skill is invoked
3. Resources	Supporting files under <code>scripts/</code> , <code>references/</code> , and <code>assets/</code>	As needed after invocation, when the instructions reference them

Skills: No todo lo que brilla es oro.

The Regression Tax: Decomposing Why Skills Help — and Hurt — LLM Agents*

Darshan Tank
darshan@sentient.xyz
Sentient Labs

Baran Nama
baran@sentient.xyz
Sentient Labs

Abstract

Adding procedural skills to an LLM agent is typically evaluated by average improvement in task success. However, this metric hides an important cost: skills can also make agents worse. We measure both sides by comparing agents with and without skills across nearly 6,000 runs spanning two office-automation benchmarks and three model-harness stacks. This allows us to distinguish two outcomes. A regression is a task solved without skills but failed after skills are added. A residual failure is a task that fails both with and without skills. We find that regressions are substantial enough that the best-performing skills outperform others primarily by regressing less, not by gaining more. We identify three causes of regression: (i) skill-description osmosis, a skill changes an agent's behavior simply by being present in context, even when it is never invoked; (ii) grounding displacement, a skill's prescribed procedure overrides how the agent interprets its inputs; and (iii) verification displacement, where the procedure suppresses checks the agent would otherwise perform on its outputs. Analysing persistent failures reveals the same underlying pattern. Existing skills overemphasize procedural guidance—the stage least often responsible for failure—while under-supporting grounding and verification, the dominant sources of remaining errors. After correcting evaluation artifacts and studying traces, we find many regressions and persistent failures recoverable through better grounding and verification.

every step and can change behavior without being called; selection-based methods do not measure that effect. We address both: why regressions happen, and how to measure presence-only effects.

We compare agents with and without skills across 5,832 task-condition runs on two office-automation benchmarks and three model-harness stacks. Within each stack, only the skill library changes. Because the pass-rate difference equals gains minus regressions divided by the number of tasks, our contribution is not a new aggregate metric but its paired decomposition: which failures a library fixes and which solved tasks it breaks. The traces suggest three regression mechanisms—*skill-description osmosis*, *grounding displacement*, and *verification displacement*—and place many residual failures at the grounding and verification stages (Figure 1).

Contributions.

- We decompose pass-rate changes into gains and regressions, showing that 324 observed regression transitions offset 59% of 553 gross gain transitions.
- We define and apply criteria for three candidate regression mechanisms, including description-only influence when no skill body is invoked.
- We locate persistent errors at grounding and verification, and re-grade 226 treatment task-conditions that the original spreadsheet grader failed.

arXiv:2407.22520 [AI] 24 Jul 2026

Skills: ¿Cómo configurar un skill?

1. Crear la carpeta de Skills

Crear una carpeta para almacenar todos los skills del proyecto.

2. Crear un Skill

Crear una subcarpeta independiente para cada skill.

3. Definir las instrucciones

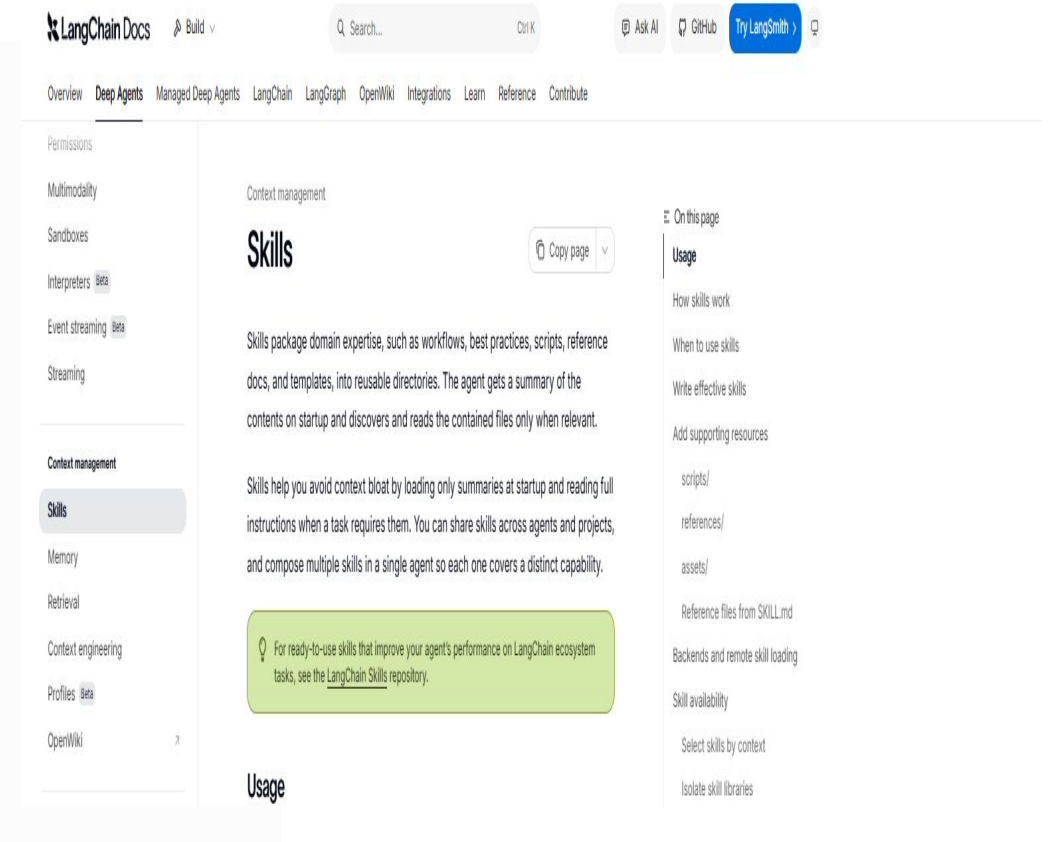
Crear el archivo `SKILL.md` con el nombre, descripción e instrucciones del skill.

4. Conectar los Skills al agente

Configurar el agente indicando dónde se encuentran los skills disponibles.

5. Ejecutar el agente

El agente identifica el skill adecuado, carga sus instrucciones y las utiliza para completar la tarea.



Nota: <https://docs.langchain.com/oss/python/deepagents/skills>

Actividad 1: ¿Cómo añadir un skill a un agente?

